GROUP 5 : LEONG ZHAN WEI, EPHRAIM CHUAH EN HAO, QIAO WENXIAO

# How well can AI detect AI?



ESSENTIAL QUESTION

# Can we reliably detect AI written text?

# DATA CHECK & CLEANING



- Dataset size
- Missing values
- Duplicate records
- Dataset info

```python
# Dataset Info
print('Dataset info:', dataset.info)

# Dataset size
print("Dataset shape:", dataset.shape)

# Missing values
print(dataset.isnull().sum())

# Duplicate records
print("Duplicate rows:", dataset.duplicated().sum())

# Data types
print(dataset.dtypes)
```

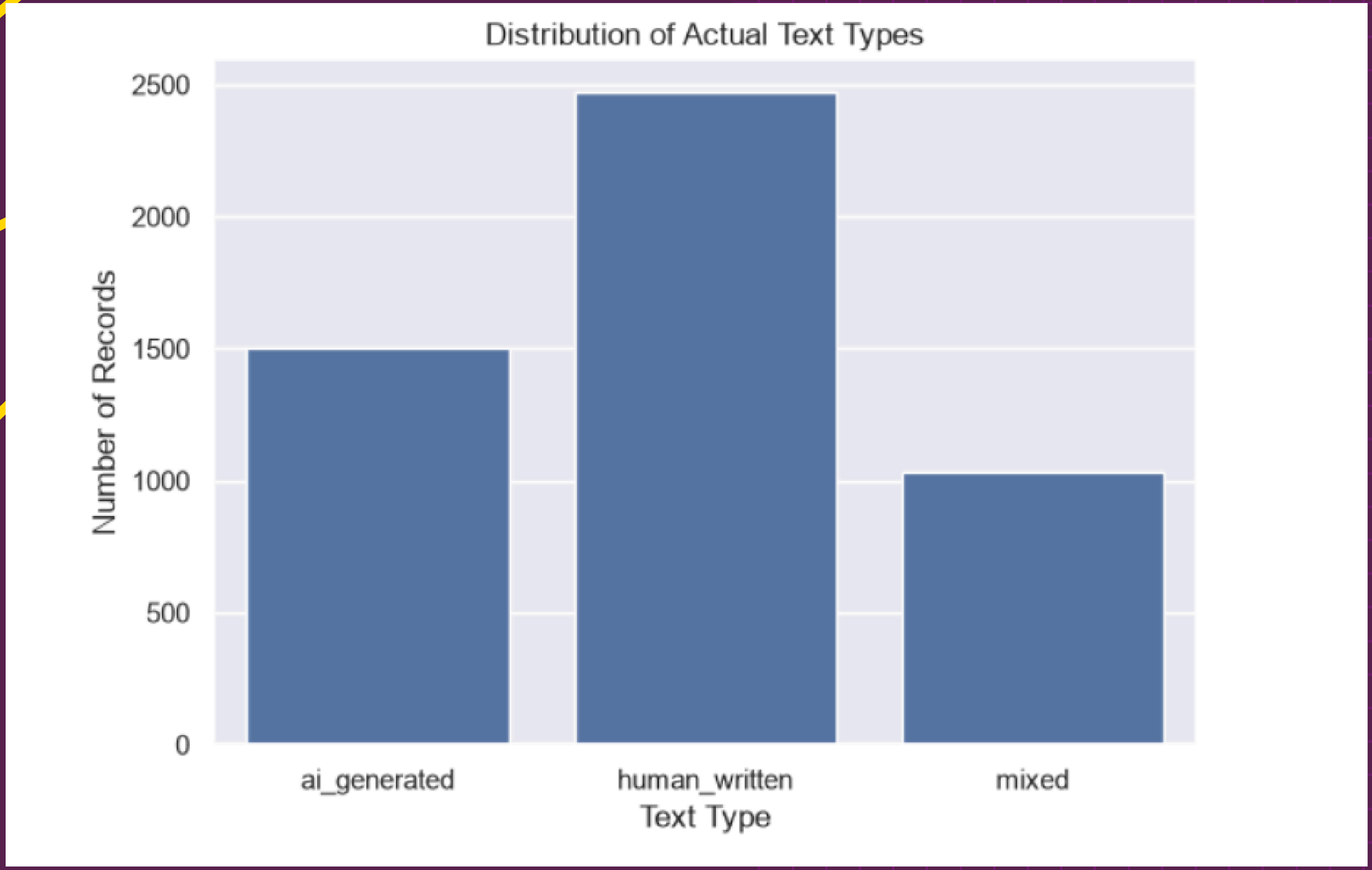
# DEFINITION



- **Perplexity Score:** predictability of words
- **Text Length:** length of the text
- **Writing Style:** how well a text convey its message & style
- **Accuracy:** proportion of correct predictions made by the model out of all predictions
- **Precision:** how many of the positive predictions made by the model are actually correct
- **Recall (True Positive Rate):** how many of the actual positive cases were correctly identified
- **F1:** harmonic mean of precision and recall
- **True Negative Rate (Specificity):** how many actual negative instances were correctly identified by the model
- **False Positive Rate:** how many actual negative instances were incorrectly classified as positive
- **False Negative Rate:** how many actual positive instances were incorrectly classified as negative

# UNDERSTANDING OUR DATASET





# UNDERSTANDING OUR DATASET



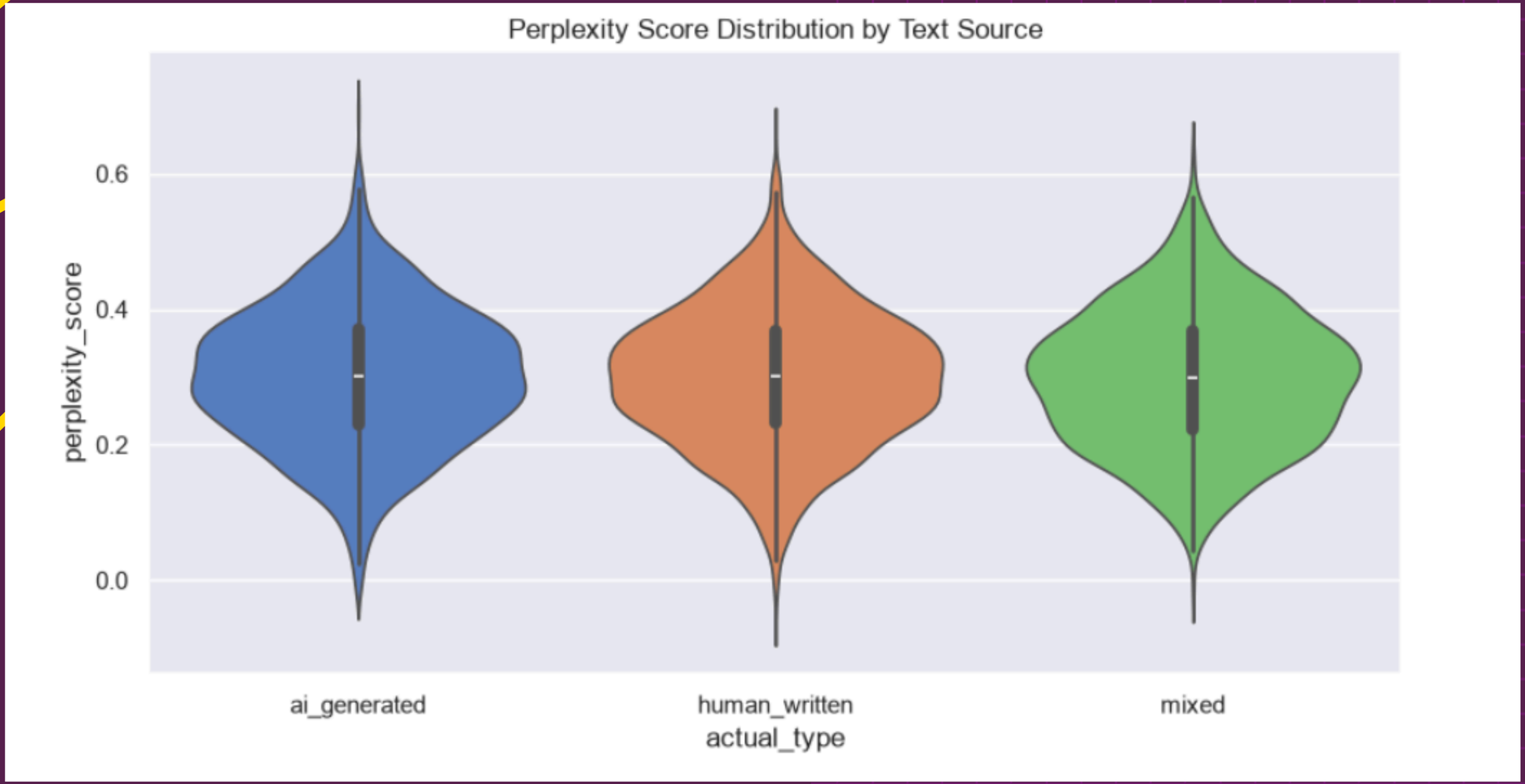
| Actual Type | Submissions | Flag Rate (%) |
|---|---|---|
| AI Generated | 1500 | 85.4 |
| Mixed | 1031 | 43.5 |
| Human Written | 2469 | 6.8 |

# UNDERSTANDING OUR DATASET



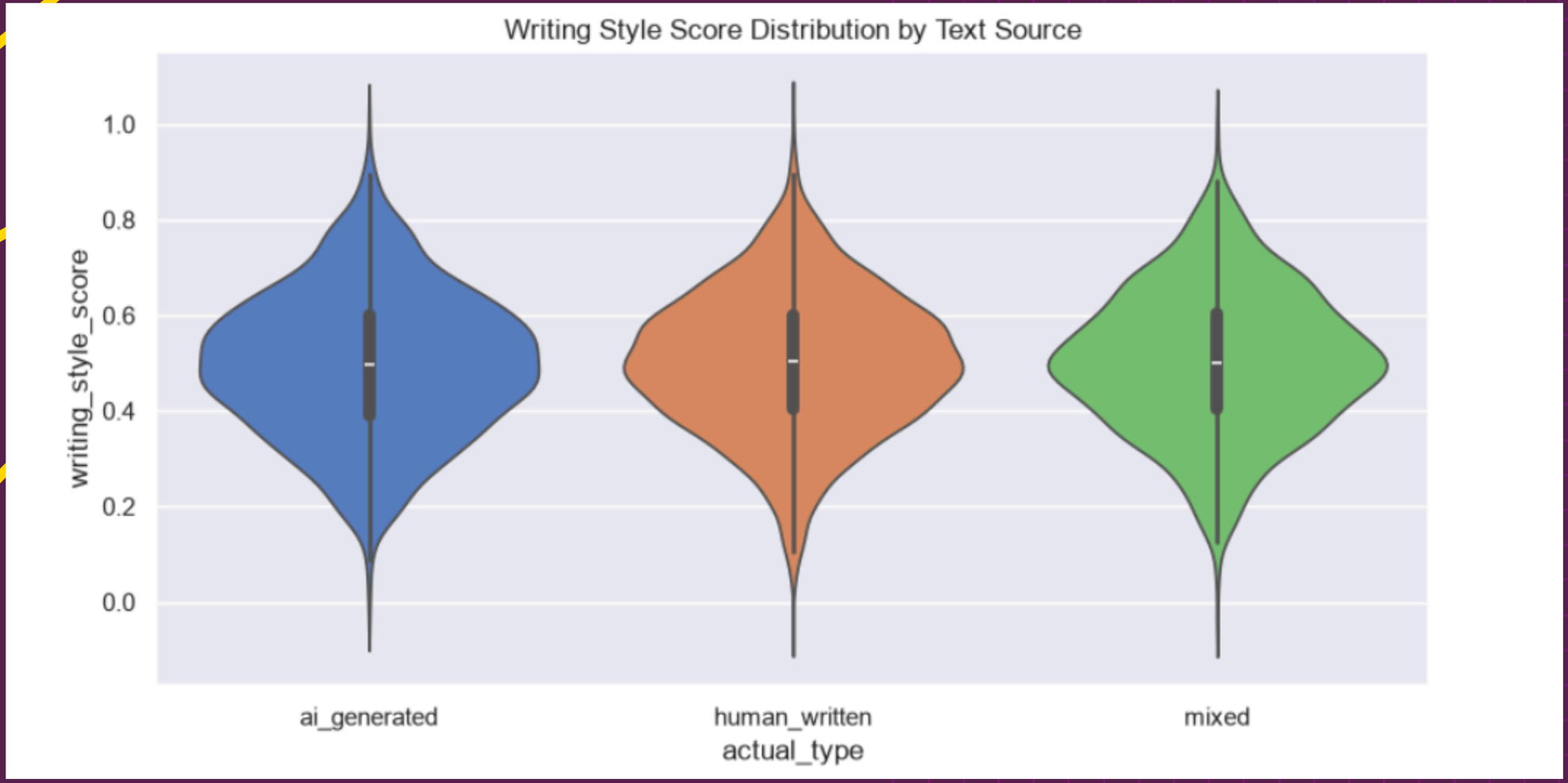
# UNDERSTANDING OUR DATASET





# UNDERSTANDING OUR DATASET





# THREE TEST (REMOVED MIXED)

1. Perplexity, Writing Style & Text Length
   Decision Tree Classifier

2. Perplexity, Writing Style, Text Length, Grade & Subject
   Decision Tree Classifier

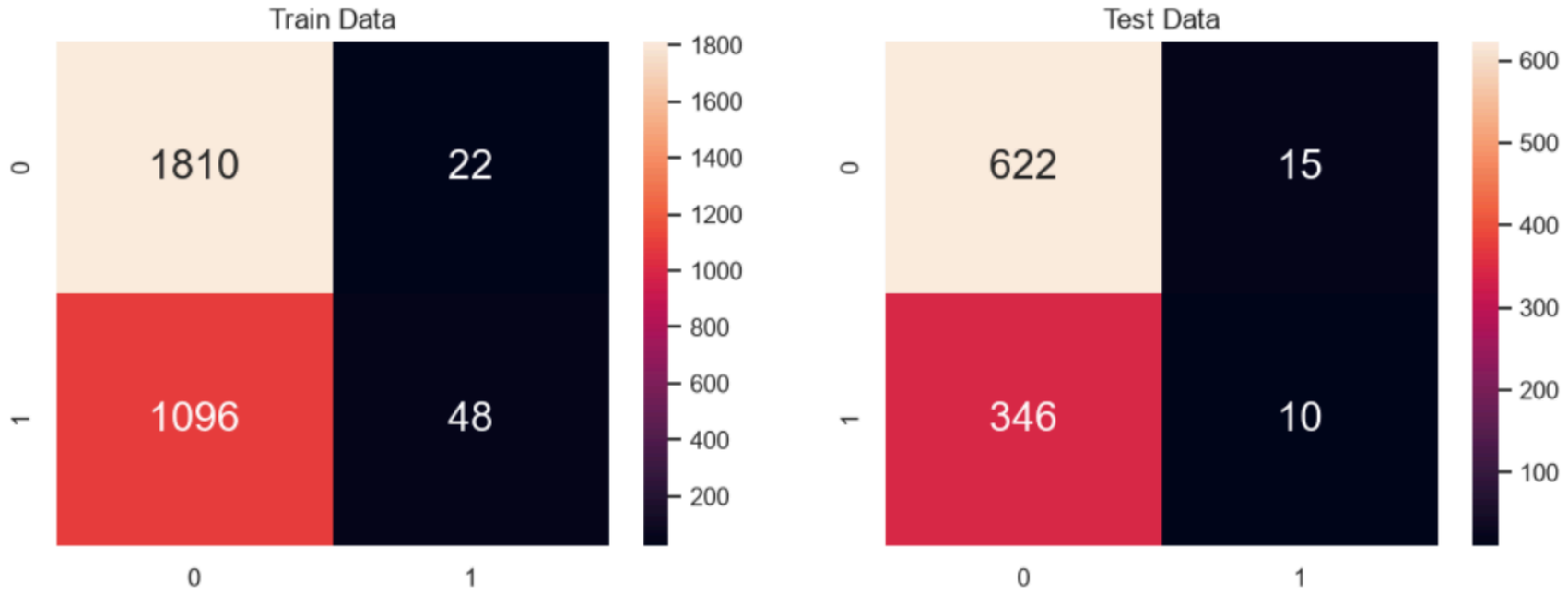
3. Perplexity, Writing Style, Text Length, Grade & Subject
   Random Forest Classifier

# Perplexity, Writing Style & Text Length

Train/Test Split: 75/25

ML model: Decision Tree Classifier (Depth 3)

# Results




```
Test Data Metrics -> Precision: 0.40 | Recall: 0.03 | F1: 0.05

Goodness of Fit of Model        Train Dataset
Classification Accuracy         : 0.6243279569892473

Goodness of Fit of Model        Test Dataset
Classification Accuracy         : 0.6364551863041289
```

# Results

Train Accuracy: 0.624 = 62.4%

Test Accuracy: 0.636 = 63.6%

Precision : 0.4

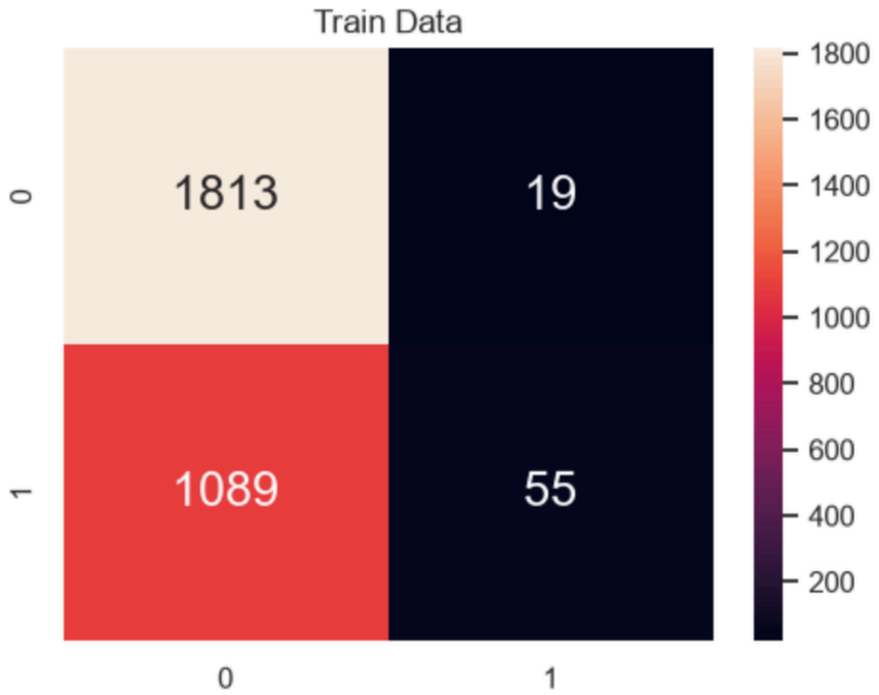
Recall : 0.03

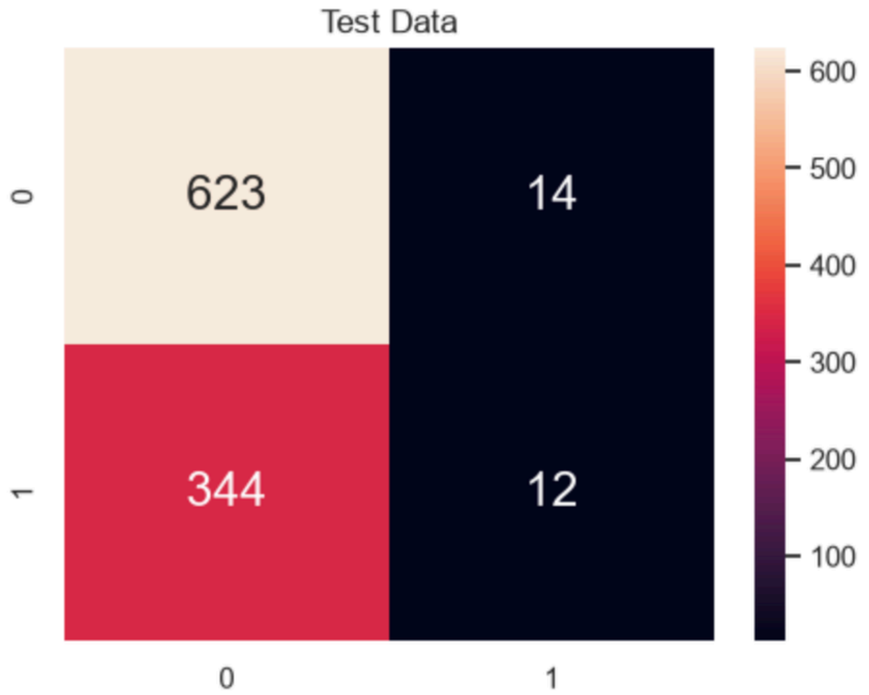
F1 : 0.05

# Perplexity, Writing Style, Text Length, Grade & Subject

Additional Features: Subject and Grade Level
Train/Test Split: 75/25%
ML Model: Decision Tree Classifier (Depth 4)

# Results




```
Test Data Metrics -> Precision: 0.46 | Recall: 0.03 | F1: 0.06

Goodness of Fit of Model        Train Dataset
Classification Accuracy         : 0.6276881720430108

Goodness of Fit of Model        Test Dataset
Classification Accuracy         : 0.6394763343403826
```

# Results

Train Accuracy: 0.627 = 62.7%

Test Accuracy: 0.636 = 63.6%

Precision : 0.46

Recall : 0.03

F1 : 0.06

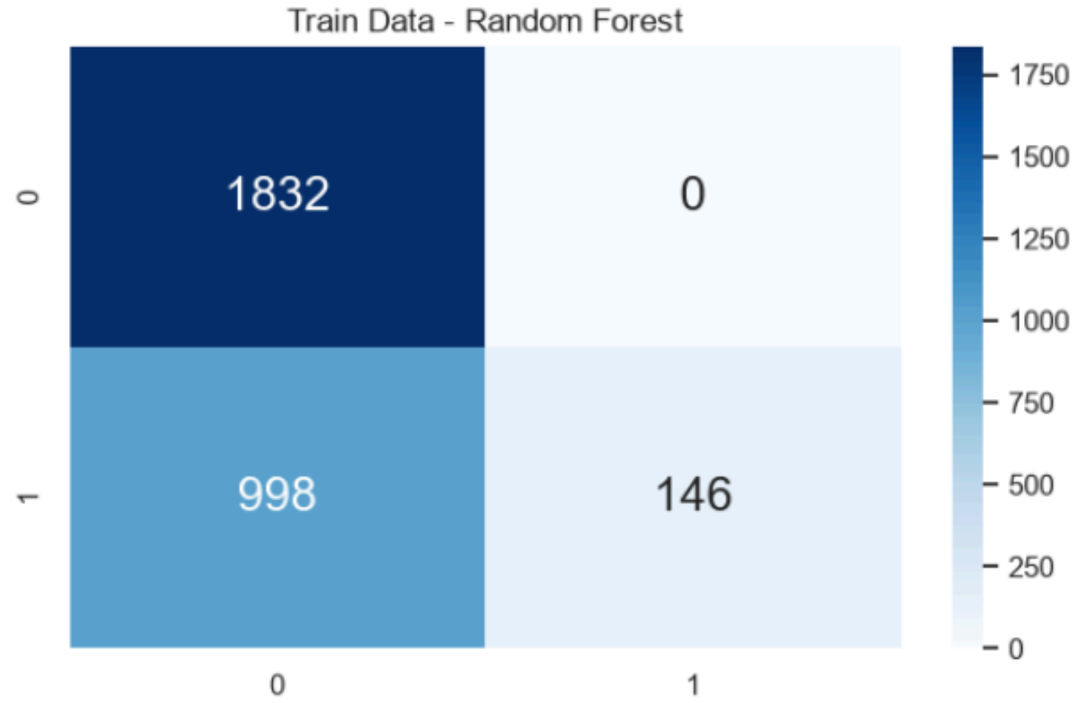
# Perplexity, Writing Style, Text Length, Grade & Subject

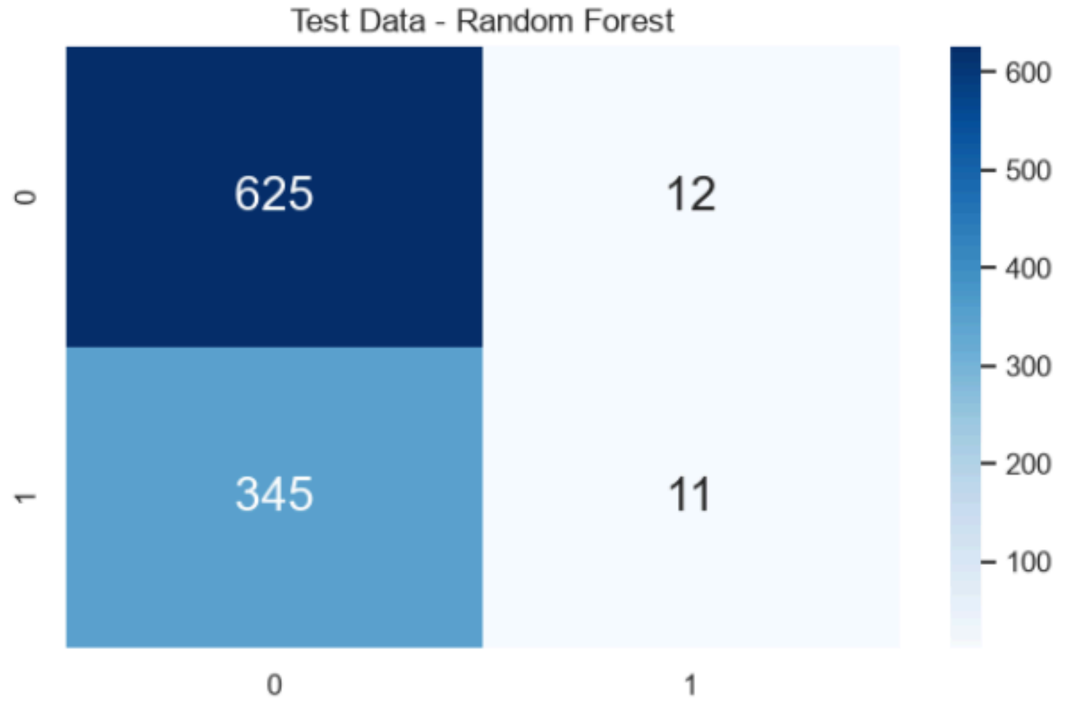
## ML Model: Random Forest Classifier

# Random Forest Classifier

- Core Concept: Combines predictions from multiple decision trees
- Eg: 1 friend’s advice vs 100 friend’s majority vote
- More accurate

# Results




```
Test Data Metrics -> Precision: 0.48 | Recall: 0.03 | F1: 0.06

Goodness of Fit of Model        Train Dataset
Classification Accuracy         : 0.6647

Goodness of Fit of Model        Test Dataset
Classification Accuracy         : 0.6405
```

# Results

| | Decision Tree | Random Forest |
|---|---|---|
| **Train Accuracy** | 62.7% | 66.5% |
| **Test Accuracy** | 63.6% | 64% |
| **Precision** | 0.46 | 0.48 |

# Conclusion

- **Random Forest Classifier performs better than Decision Tree Classifier**
- **Text characteristics metrics alone are insufficient**
- **Deeper mechanics involved**

# Which AI Tool performs best?

common sense media

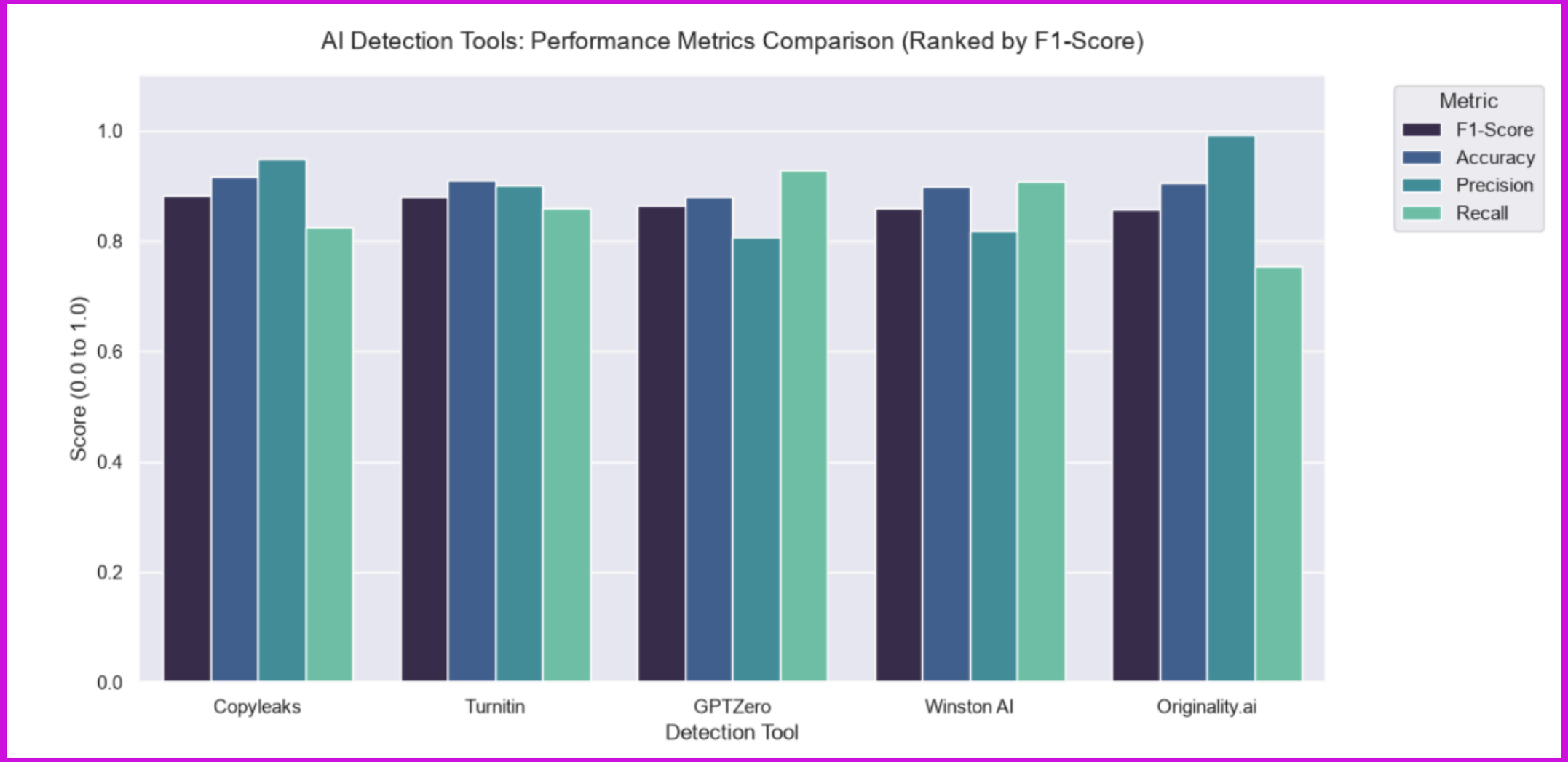
# The Test

1. **Compare each AI model's decision (detected_as_ai) vs the actual type of the work (actual_type)**
2. **Calculate accuracy, f1, recall and precision scores for each AI model.**

# Results

| Rank | AI Tool | F1 | Accuracy | Precision | Recall |
|---|---|---|---|---|---|
| 1 | Copyleaks | 0.883 | 91.7% | 95% | 82.6% |
| 2 | Turnitin | 0.880 | 90.9% | 90.1% | 85.9% |
| 3 | GPTZero | 0.863 | 88.1% | 80.7% | 92.7% |
| 4 | Winston AI | 0.861 | 89.9% | 81.8% | 90.8% |
| 5 | Originality.ai | 0.856 | 90.5% | 99.1% | 75.3% |

# Results


AI Detection Tools: Performance Metrics Comparison (Ranked by F1-Score)
Score (0.0 to 1.0)
1.0
0.8
0.6
0.4
0.2
0.0
Copyleaks
Turnitin
GPTZero
Winston AI
Originality.ai
Detection Tool
Metric
F1-Score
Accuracy
Precision
Recall

# Conclusion

Copyleaks performs the best overall in predicting AI (Chatgpt) generated or human written work!

# Limitations of our test

- **Dataset limitations:** Our results may not reflect how the models perform on new writing samples
- **Mixed text is harder:** Texts combining human and AI writing were harder to classify (what percentage is considered AI?)
- **Perfect accuracy does not guarantee reliability:** AI detector scores may contain strong patterns specific to the dataset

# Conclusion

**Can We Reliably Detect AI-Written Text?**

**1. Text characteristics alone were insufficient.**

**Our initial models achieved approximately 64% accuracy and struggled to identify AI-generated samples.**

**2. AI detection scores greatly improved predictions.**

**Models using detection probability and confidence score achieved very high accuracy on our dataset.**

**3. Machine learning has limitations.**

**High accuracy on one dataset does not guarantee that a model will perform equally well in real-world situations.**

**AI detection can be a useful supporting tool, but should not be treated as definitive proof of who wrote a text.**

# Conclusion

4. **Random Forest Classifier performs better than Decision Tree Classifier as it prevents overfitting (accurate train, inaccurate test) and reduces variants via majority vote**
5. **Copyleaks performs the best overall in predicting AI (Chatgpt) generated or human written work! (Limitations: Dataset bias & model decay)**

**AI detection can be a useful supporting tool, but should not be treated as definitive proof of who wrote a text.**

# Thanks for listening!!

# Reference

**Gemini:**

- **Chose Random Forest Classifier, and came out with the best parameters (n_estimators = 100, max_depth = 3)**
- **Come out with the score metrics (precision, recall & f1)**
- **For loop, to compare detected_as_ai vs actual_type for every AI tool, and drawing the bar chart for visualization**
- **Conclusion ideas (we changed a bit to further suit our study)**

**Definitions:**
**https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/**

# Code

## Test 1 & 2

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix


X = dataset_binary[['perplexity_score', 'writing_style_score', 'text_length']]
y = dataset_binary['actual_type']

# Added random_state for reproducible splits
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 42)

dectree = DecisionTreeClassifier(max_depth = 3)
dectree.fit(X_train, y_train)

y_train_pred = dectree.predict(X_train)
y_test_pred = dectree.predict(X_test)

f, axes = plt.subplots(1, 2, figsize=(12, 4))
sb.heatmap(confusion_matrix(y_train, y_train_pred, labels=['human_written', 'ai_generated']),
           annot = True, fmt=".0f", annot_kws={"size": 18}, ax = axes[0])
axes[0].set_title('Train Data')

sb.heatmap(confusion_matrix(y_test, y_test_pred, labels=['human_written', 'ai_generated']),
           annot = True, fmt=".0f", annot_kws={"size": 18}, ax = axes[1])
axes[1].set_title('Test Data')
plt.show()


cm = confusion_matrix(y_test, y_test_pred, labels=['human_written', 'ai_generated'])

# .ravel() flattens the 2x2 matrix into 4 individual variables
TN, FP, FN, TP = cm.ravel()

# Manual Formulas
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0

print(f"Test Data Metrics -> Precision: {precision:.2f} | Recall: {recall:.2f} | F1: {f1:.2f}\n")

print("Goodness of Fit of Model \tTrain Dataset")
print("Classification Accuracy \t:", dectree.score(X_train, y_train))
print()

print("Goodness of Fit of Model \tTest Dataset")
print("Classification Accuracy \t:", dectree.score(X_test, y_test))
```

```
# Data Preprocessing
dataset_binary = dataset[dataset['actual_type'].isin(['ai_generated', 'human_written'])].copy()

# Convert categorical text columns into 1s and 0s
dataset_binary = pd.get_dummies(dataset_binary, columns=['subject', 'grade_level'], drop_first=True)

# Now include all these new columns in X
X = dataset_binary.drop(columns=['submission_id', 'tool', 'actual_type', 'detected_as_ai', 'confidence_score', 'detection_probability'])
y = dataset_binary['actual_type']

# Added random_state for reproducible splits
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 42)

dectree = DecisionTreeClassifier(max_depth = 4)
dectree.fit(X_train, y_train)

y_train_pred = dectree.predict(X_train)
y_test_pred = dectree.predict(X_test)

f, axes = plt.subplots(1, 2, figsize=(12, 4))
sb.heatmap(confusion_matrix(y_train, y_train_pred, labels=['human_written', 'ai_generated']),
           annot = True, fmt=".0f", annot_kws={"size": 18}, ax = axes[0])
axes[0].set_title('Train Data')

sb.heatmap(confusion_matrix(y_test, y_test_pred, labels=['human_written', 'ai_generated']),
           annot = True, fmt=".0f", annot_kws={"size": 18}, ax = axes[1])
axes[1].set_title('Test Data')
plt.show()

cm = confusion_matrix(y_test, y_test_pred, labels=['human_written', 'ai_generated'])

# .ravel() flattens the 2x2 matrix into 4 individual variables
TN, FP, FN, TP = cm.ravel()

# Manual Formulas
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0

print(f"Test Data Metrics -> Precision: {precision:.2f} | Recall: {recall:.2f} | F1: {f1:.2f}\n")

print("Goodness of Fit of Model \tTrain Dataset")
print("Classification Accuracy \t:", dectree.score(X_train, y_train))
print()

print("Goodness of Fit of Model \tTest Dataset")
print("Classification Accuracy \t:", dectree.score(X_test, y_test))
```

# Code

## Test 3 & Study 2

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from IPython.display import display

# 1. Load and Prepare the Dataset
dataset = pd.read_csv('chatgpt_detection_results.csv')

# Filter out 'mixed' text to create a strict Human vs. AI test
dataset = dataset[dataset['actual_type'] != 'mixed'].copy()

# Map strings to binary numbers (1 for AI, 0 for Human)
dataset['is_ai_actual'] = (dataset['actual_type'] == 'ai_generated').astype(int)
dataset['is_ai_predicted'] = dataset['detected_as_ai'].astype(int)

# 2. Calculate Metrics for Each Tool
results = []
for tool in dataset['tool'].unique():
    tool_data = dataset[dataset['tool'] == tool]

    y_true = tool_data['is_ai_actual']
    y_pred = tool_data['is_ai_predicted']

    results.append({
        'Tool': tool,
        'F1-Score': f1_score(y_true, y_pred, zero_division=0),
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, zero_division=0),
        'Recall': recall_score(y_true, y_pred, zero_division=0)
    })

# 3. Format and Rank the Data
results_df = pd.DataFrame(results).sort_values(by='F1-Score', ascending=False)
results_df.insert(0, 'Rank', range(1, len(results_df) + 1))

# Create a display copy to format strings as percentages for the text table
display_df = results_df.copy()
display_df['F1-Score'] = display_df['F1-Score'].apply(lambda x: f"{x:.3f}")
display_df['Accuracy'] = display_df['Accuracy'].apply(lambda x: f"{x*100:.1f}%")
display_df['Precision'] = display_df['Precision'].apply(lambda x: f"{x*100:.1f}%")
display_df['Recall'] = display_df['Recall'].apply(lambda x: f"{x*100:.1f}%")

# Print the leaderboard using IPython display for clean formatting in Jupyter
print("--- AI Detection Tool Performance Leaderboard ---")
display(display_df.style.hide(axis="index"))

# 4. Draw the Visual Chart
# Melt converts the distinct metric columns into a single 'Metric' column for Seaborn
melted_df = results_df.drop(columns=['Rank']).melt(id_vars='Tool', var_name='Metric', value_name='Score')

plt.figure(figsize=(12, 6))

sns.barplot(
    data=melted_df,
    x='Tool',
    y='Score',
    hue='Metric',
    palette='mako'
)

plt.title('AI Detection Tools: Performance Metrics Comparison (Ranked by F1-Score)', fontsize=14, pad=15)
plt.ylim(0, 1.1)
plt.ylabel('Score (0.0 to 1.0)', fontsize=12)
plt.xlabel('Detection Tool', fontsize=12)

# Move the legend outside the chart so it doesn't block the bars
plt.legend(title='Metric', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sb
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, classification_report

# 1. Data Preprocessing
dataset_binary = dataset[dataset['actual_type'].isin(['ai_generated', 'human_written'])].copy()

# Convert categorical text columns into 1s and 0s
dataset_binary = pd.get_dummies(dataset_binary, columns=['subject', 'grade_level'], drop_first=True)

# Define X and y
X = dataset_binary.drop(columns=['submission_id', 'tool', 'actual_type', 'detected_as_ai', 'confidence_score', 'detection_probability'])
y = dataset_binary['actual_type']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# 2. Train the Random Forest Model
# n_estimators=100 means we are building a "forest" of 100 decision trees
rf_model = RandomForestClassifier(n_estimators=100, max_depth=8, random_state=42)
rf_model.fit(X_train, y_train)

# 3. Generate Predictions
y_train_pred = rf_model.predict(X_train)
y_test_pred = rf_model.predict(X_test)

# 4. Visualize with Confusion Matrices
f, axes = plt.subplots(1, 2, figsize=(12, 4))
sb.heatmap(confusion_matrix(y_train, y_train_pred, labels=['human_written', 'ai_generated']),
           annot=True, fmt=".0f", annot_kws={"size": 18}, ax=axes[0], cmap="Blues")
axes[0].set_title('Train Data - Random Forest')

sb.heatmap(confusion_matrix(y_test, y_test_pred, labels=['human_written', 'ai_generated']),
           annot=True, fmt=".0f", annot_kws={"size": 18}, ax=axes[1], cmap="Blues")
axes[1].set_title('Test Data - Random Forest')
plt.tight_layout()
plt.show()

# 5. Model Evaluation
cm = confusion_matrix(y_test, y_test_pred, labels=['human_written', 'ai_generated'])
TN, FP, FN, TP = cm.ravel()

# Manual Formulas
precision = TP / (TP + FP) if (TP + FP) > 0 else 0.0
recall = TP / (TP + FN) if (TP + FN) > 0 else 0.0
f1 = (2 * precision * recall) / (precision + recall) if (precision + recall) > 0 else 0.0

print(f"Test Data Metrics -> Precision: {precision:.2f} | Recall: {recall:.2f} | F1: {f1:.2f}\n")

print("Goodness of Fit of Model \tTrain Dataset")
print(f"Classification Accuracy \t: {rf_model.score(X_train, y_train):.4f}\n")

print("Goodness of Fit of Model \tTest Dataset")
print(f"Classification Accuracy \t: {rf_model.score(X_test, y_test):.4f}")
```